



when the code is modern

java has primitives i guess?

Type	Size	Range	Default
byte	8 bits	-128 to 127	0
short	16 bits	-32,768 to 32,767	0
int	32 bits	-2.1B to 2.1B	0
long	64 bits	-9.2Q to 9.2Q	0L

Type	Size	Range	Default
float	32 bits	$\pm 3.4\text{e}38$	0.0f
double	64 bits	$\pm 1.8\text{e}308$	0.0

Type	Size	Range	Default
boolean	1 bit	true/false	false
char	16 bits	0 to 65,535	\u0000

short circuit (littstore.com)

- **Integer division**: When both operands are **int**, result is **int**
- **Floating-point division**: When one operand is **double**, result is **double**
- **Type promotion**: Java automatically widens **int** to **double** when needed
- **Precision loss**: Integer division truncates (cuts off) the decimal part

```
int a = 3 / 2; // 1 (int division)
double b = 3 / 2; // 1.0 (int division, then widened)
double c = 3 / 2.0; // 1.5 (floating division)
double d = 3.0 / 2; // 1.5 (floating division)
```

Precedence	Operators	Description
1 (Highest)	() [] .	Parentheses, brackets, dot
2	++ --	Postfix increment/decrement
3	++ -- + - ! ~	Prefix increment/decrement, unary
4	* / %	Multiplication, division, modulo
5	+ -	Addition, subtraction
6	< > >>	Bitwise shifts
7	< <= > >=	Relational operators
8	== !=	Equality operators
9	&	Bitwise AND
10	^	Bitwise XOR
11		Bitwise OR
12	&&	Logical AND
13		Logical OR
14 (Lowest)	?: += -=	Conditional, assignment

!(A && B) is the same as (!A || !B)

Loop Control: When to Use break & continue

break examples:

- **Search loops**: Found the item, stop looking
- **Game loops**: Player won/lost, end the game
- **Error handling**: Something went wrong, stop processing
- **Input validation**: Got valid input, no need to ask again

continue examples:

- **Data filtering**: Skip invalid or unwanted items
- **Processing**: Skip items that don't need full processing
- **Error recovery**: Skip problematic items, continue with others

Real-world analogy:

- **break**: Like finding your keys and stopping the search
- **continue**: Like skipping full parking spots while looking for an empty one

- **Access modifiers**: control who can call the method
 - **public**: anyone can call this method (most permissive)
 - **private**: only code in the same class can call this method
 - **protected**: only code in the same package or subclasses can call
 - **(default)**: only code in the same package can call (no modifier written)
- **Behavior modifiers**: change how the method works
 - **static**: method belongs to the class, not to instances
 - **final**: method cannot be overridden by subclasses
 - **synchronized**: method is thread-safe (advanced topic)

- **Short-circuit evaluation**: second operand may not be evaluated

```
1 boolean hasKey = true;
2 boolean hasWeapon = false;
3 boolean isAlive = true;
4
5 boolean canOpenDoor = hasKey && isAlive; // true
6 boolean canFight = hasWeapon || isAlive; // true
7 boolean isDead = !isAlive; // false
```

Autoboxing: int <-> Integer

- **Autoboxing**: Java automatically converts between primitives and wrappers
- **Boxing**: int -> Integer (automatic)
- **Unboxing**: Integer -> int (automatic)

```
1 // Autoboxing examples
2 int primitive = 42;
3 Integer wrapper = primitive; // int -> Integer (boxing)
4 int backToPrimitive = wrapper; // Integer -> int (unboxing)
5 // Mixed operations
6 Integer score = 100; // Integer object
7 int bonus = 25; // int primitive
8 Integer total = score + bonus; // Integer + int = Integer (boxing)
9 int total2 = score + bonus; // Integer + int = int (unboxing)
```

When to use break & continue:

- **break**: When you need to exit early (found target, error occurred)
- **continue**: When you want to skip current iteration but continue looping

When NOT to use them:

- **Complex logic**: Makes code harder to read and debug
- **Multiple levels**: Can make nested loops confusing
- **Overuse**: Can indicate poor loop design

Better alternatives:

- **Clear conditions**: Design loops with natural exit conditions
- **Boolean flags**: Use variables to control loop behavior
- **Separate functions**: Extract complex logic into helper methods

Remember: These are tools, not crutches. Use them when they make code clearer!

JVM Memory Areas: Where Data Lives

- **Java runs inside the JVM** which organizes memory into distinct areas
- **Method Area**: Permanent storage for class information
 - Class definitions, method code, static variables
 - Constants and metadata
- **Stack**: Temporary storage for method execution
 - Method calls, parameters, local variables
 - Each method gets its own "frame"
- **Heap**: Dynamic storage for objects
 - Object instances, arrays, dynamic data
 - Managed by garbage collection
- **PC Register**: Points to the current instruction being executed
- **Native Method Stack**: For native methods (code written in other languages)



67



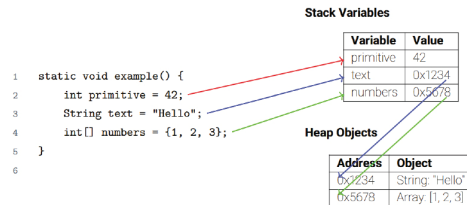


- **Stack structure:** Last In, First Out (LIFO) - like a stack of plates
- **Stack frame:** Each method call gets its own "plate" containing:
 - Method parameters (copied from caller)
 - Local variables declared in the method
 - Return address (where to go when method finishes)
 - Temporary calculation results
- **Automatic management:** Frames are created when methods start, destroyed when they end
- **Characteristics:** Limited size, very fast access, no fragmentation, automatic cleanup
- **Call chain:** `main()` → `process()` → `helper()`
- **What happens:**
 - Each method call creates a new stack frame
 - Parameters are copied from caller's frame to callee's frame
 - Local variables are created in the new frame
 - When method returns, its frame is destroyed
- **Stack depth:** Maximum number of frames = maximum call depth
- **Why it matters:** Stack overflow happens when call chain is too deep



Stack vs Heap: Visual Example

- **Heap structure:** Large, flexible memory area for objects
- **What lives here:**
 - All objects created with `new` keyword
 - Arrays (both primitive and object arrays)
 - String objects and other reference types
 - Large data structures
- **Memory management:** Garbage collector frees unused objects
- **Access:** Through references (memory addresses) stored in variables
- **Characteristics:** Much larger than stack, can grow/shrink, potential fragmentation, slower access

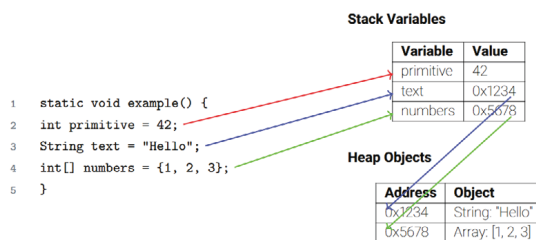


Design Checklist

Design Benefits of Method Signatures

- **Interface First:** Define how methods interact before implementation
- **Parameter Clarity:** Forces you to think about what data is needed
- **Return Type Planning:** Clarifies what each method produces
- **Testing Strategy:** You can test method contracts before implementation
- **Team Collaboration:** Others can understand your design immediately
- **Test each method in isolation:** with simple test cases
- **Cover edge cases:** 0, negative numbers, boundary conditions
- **Use assertions** to verify results automatically
- **Test the complete system** with known inputs and expected outputs
- **Start testing early** - easier to fix problems when methods are small
- **Test scope boundaries:** Ensure methods don't access unintended variables
- **Test memory model understanding:** Verify parameter passing behavior
- **Single purpose per method:** Each method does one clear thing
- **Verb names:** Methods start with action words (compute, apply, render)
- **Clear inputs/outputs:** Parameters provide data, return values communicate results
- **Keep shared mutable state out:** Prefer parameters and returns over global variables
- **Choose types thoughtfully:** `int` vs `double`, document units clearly
- **Test helpers in isolation:** Each method should work independently
- **Scope awareness:** Methods should not access variables outside their scope
- **Assertions:** Automatic checks that fail if conditions aren't met
- **Java Assertions:** Disabled by default, enable with `-ea` flag
- **Given-When-Then Pattern:**
 - **Given:** Set up test data and initial conditions
 - **When:** Execute the method being tested
 - **Then:** Verify the results with assertions
- **Benefits:** Tests are self-documenting and fail fast
- **Example:** Given base damage 10, When computing with 1.5x crit, Then result should be 15

Reminder: Arrays live on the heap



`int[] scores = new int[5];`

arrays use `.length` idiot!



67



searching

for loaf <3

linear search is a thing too

Binary Search: The Main Loop

```
1 while (left <= right) {
2     int mid = (left + right) / 2;
3     if (sorted[mid] == target) {
4         index = mid;
5         break;
6     } else if (sorted[mid] < target) {
7         left = mid + 1;
8     } else {
9         right = mid - 1;
10    }
11 }
```

```
// Method 1
// Recursive binary search
// Returns index of x if it is present
// in arr[l..r], else return -1
int binarySearch(int arr[], int l, int r, int x)
{
    // Restrict the boundary of right index
    // and the left index to prevent
    // overflow of indices
    if (r >= 1 && l <= arr.length - 1) {
        int mid = l + (r - l) / 2;

        // If the element is present
        // at the middle itself
        if (arr[mid] == x)
            return mid;

        // If element is smaller than mid, then it can
        // only be present in left subarray
        if (arr[mid] > x)
            return binarySearch(arr, l, mid - 1, x);

        // Else the element can only be present
        // in right subarray
        return binarySearch(arr, mid + 1, r, x);
    }

    // We reach here when element is not present in
    // array
    return -1;
}
```

$\log_2(1e6) \approx 20$

Sorting

Memory Usage

- **In-place:** Sort within the original array (uses 1 extra space)
- **Out-of-place:** Create new array for sorted result (uses n extra spaces)

Stability

- **Stable:** Preserves relative order of equal elements
- **Unstable:** May change relative order of equal elements

Adaptive

- **Adaptive:** Performance improves on partially sorted data
- **Non-adaptive:** Performance is the same regardless of input order

Memory Usage

- **In-place:** Selection, Bubble, Insertion, Shaker, Quick, Heap, Shell
- **Out-of-place:** Merge Sort

Stability

- **Stable:** Bubble, Insertion, Merge, Shaker
- **Unstable:** Selection, Quick, Heap, Shell

Adaptive

- **Yes:** Bubble, Insertion, Shaker
- **No:** Selection, Quick, Merge, Heap, Shell

Simple sorts (easy to understand and implement)

- **Bubble Sort:** Compare neighbors, bubble up
- **Selection Sort:** Find minimum, put in front
- **Insertion Sort:** Insert each element in right place
- **Shaker Sort:** Bidirectional bubble sort

Advanced sorts (better performance for large arrays)

- **Merge Sort:** Divide array, sort halves, merge
- **Quick Sort:** Pick pivot, partition, sort recursively
- **Shell Sort:** Sort with decreasing gaps
- **Heap Sort:** Build heap, extract maximums

Bubble Sort: Core Concept

Strategy: Compare and swap adjacent elements

- **Key idea:** "Bubble" the largest element to the end in each pass
- **Process:** Compare each pair of adjacent elements, swap if out of order
- **Result:** After each pass, the largest unsorted element reaches its final position
- **Optimization:** Stop early if no swaps occur (array is sorted)

```
void bubbleSort(int[] a) {
    boolean swapped;
    for (int pass = 0; pass < a.length - 1; pass++) {
        swapped = false;
        for (int i = 0; i < a.length - 1 - pass; i++) {
            if (a[i] > a[i + 1]) {
                swap(a, i, i + 1);
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted
    }
}
```

Selection Sort: Core Concept

Strategy: Find the minimum element and place it at the beginning

- **Key idea:** In each iteration, find the smallest unsorted element
- **Process:** Scan the unsorted portion, identify minimum, swap to correct position
- **Result:** After each iteration, one more element is in its final position
- **Invariant:** Elements before current position are sorted

```
void selectionSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < a.length; j++) {
            if (a[j] < a[minIndex]) minIndex = j;
        }
        if (minIndex != i) {
            swap(a, i, minIndex);
        }
    }
}
```

Insertion Sort: Core Concept

Strategy: Insert each element into its correct position in the sorted portion

- **Key idea:** Build the sorted array one element at a time
- **Process:** Take next element, shift larger elements right, insert in correct spot
- **Result:** After each insertion, the left portion remains sorted
- **Invariant:** Elements to the left of current position are sorted

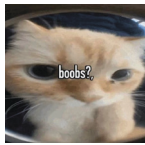
```
void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```



67



do i really put all sorting
algs here? yea.



Shaker Sort: Core Concept

Strategy: Bidirectional bubble sort - bubble in both directions

- **Key idea:** Alternate between forward and backward passes
- **Process:** Forward pass bubbles largest to end, backward pass bubbles smallest to beginning
- **Result:** Both ends get sorted simultaneously, reducing total passes needed
- **Optimization:** Stop when no swaps occur in either direction

Concept: Forward: [64, 34, 25, 12, 22, 11] → [34, 25, 12, 22, 11, 64]

Backward: [34, 25, 12, 22, 11, 64] → [11, 34, 25, 12, 22, 64]

Merge Sort: Core Concept

Strategy: Divide array in half, sort each half, merge results

- **Key idea:** Split problem into smaller identical problems
- **Process:** Recursively sort left and right halves, then merge them
- **Result:** After merging, the combined array is sorted
- **Stable:** Preserves relative order of equal elements

```
1 void mergeSort(int[] arr, int left, int right) {
2     if (left < right) {
3         int mid = (left + right) / 2;
4         mergeSort(arr, left, mid);
5         mergeSort(arr, mid + 1, right);
6         merge(arr, left, mid, right);
7     }
8 }
```

- **Base case:** If left >= right, array has 0 or 1 elements (already sorted)
- **Divide:** Split array into two halves at middle position
- **Conquer:** Recursively sort both halves
- **Combine:** Merge the sorted halves back together

Quick Sort: Core Concept

Strategy: Pick a pivot element, partition around it, sort partitions recursively

- **Key idea:** Choose a "pivot" and arrange elements around it
- **Process:** Partition array so all elements < pivot are left, > pivot are right
- **Result:** Pivot is in final position, sort left and right partitions
- **In-place:** Sorts array without additional memory

The Quick Sort Process:

1. **Base case:** If low >= high, array has 0 or 1 elements (already sorted)
2. **Partition:** Rearrange array so pivot is in correct position
3. **Recurse:** Sort left and right partitions separately
4. **Result:** Pivot is in final position, partitions are sorted

Key insight: Each partition step puts one element (the pivot) in its final sorted position, then we recursively sort the remaining elements.

```
void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

How Partition Works:

1. **Choose pivot:** Use last element as pivot
2. **Track position:** i marks end of "smaller than pivot" section
3. **Move elements:** Swap smaller elements to left of pivot position
4. **Place pivot:** Put pivot in its final sorted position

Key insight: After partitioning, the pivot is in its final sorted position, and all elements to its left are smaller, all to its right are larger.

```
int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr, i, j);
        }
    }
    swap(arr, i + 1, high);
    return i + 1;
}
```

```
void shellSort(int[] arr) {
    int n = arr.length;
    // Start with large gap, reduce by half each time
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) { // Insertion sort
            int temp = arr[i];
            int j;
            for (j = i; j > gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```

Shell Sort: Core Concept

Strategy: Sort with decreasing gaps between compared elements

- **Key idea:** Start with large gaps, reduce gap size until gap = 1
- **Process:** Use insertion sort with different gap sizes
- **Result:** Early passes move elements closer to final positions
- **In-place:** Sorts array without additional memory

The Shell Sort Process:

1. **Start with large gap:** Begin with gap = n/2
2. **Insertion sort with gap:** Sort elements that are gap positions apart
3. **Reduce gap:** Divide gap by 2 each iteration
4. **Repeat until gap = 1:** Final pass is standard insertion sort

Early passes with large gaps move elements much closer to their final positions, making the final insertion sort very efficient.

```
1 void shakerSort(int[] a) {
2     boolean swapped = true;
3     int start = 0, end = a.length - 1;
4     while (swapped) {
5         swapped = false;
6         // Forward pass
7         for (int i = start; i < end; i++) {
8             if (a[i] > a[i + 1]) {
9                 swap(a, i, i + 1);
10                swapped = true;
11            }
12        }
13        end--;
14        // Backward pass
15        for (int i = end; i > start; i--) {
16            if (a[i] < a[i - 1]) {
17                swap(a, i, i - 1);
18                swapped = true;
19            }
20        }
21        start++;
22    }
23 }
```

Merge Function: How It Works

Goal: Combine two sorted arrays into one sorted array

The merge process:

1. **Create temporary arrays** to hold the two sorted halves
2. **Copy data** from original array to temp arrays
3. **Compare and merge** elements from both temp arrays
4. **Handle remaining** elements from either array

Why temp arrays?

- Allows us to merge back into the original array safely
- Prevents overwriting data we still need to read
- Makes the algorithm easier to understand and implement

```
void merge(int[] arr, int left, int mid, int right) {
    // Create temporary arrays
    int[] leftArr = new int[mid - left + 1];
    int[] rightArr = new int[right - mid];

    // Copy data to temp arrays
    for (int i = 0; i < leftArr.length; i++) {
        leftArr[i] = arr[left + i];
    }
    for (int j = 0; j < rightArr.length; j++) {
        rightArr[j] = arr[mid + 1 + j];
    }

    // Merge the temp arrays back
    int i = 0, j = 0, k = left;
    while (i < leftArr.length && j < rightArr.length) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        } else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements
    while (i < leftArr.length) arr[k++] = leftArr[i++];
    while (j < rightArr.length) arr[k++] = rightArr[j++];
}
```

Heap Sort: Core Concept

Strategy: Build a heap, repeatedly extract the maximum element

- **Key idea:** Use heap data structure to efficiently find maximum
- **Process:** Build max-heap, swap root with last element, heapify
- **Result:** After each extraction, largest element goes to end
- **In-place:** Sorts array without additional memory

```
// Example: Sorting [64, 34, 25, 12, 22, 11, 90]
// Step 1: Build max-heap
// 90
// 64 34
// 25 12
// 22 11
// 12 22 11 90

// Step 2: Extract max (90), swap with last element
// [64, 34, 25, 12, 22, 11, 90] -> [64, 34, 25, 12, 22, 11, 90]

// Step 3: Heapify remaining elements, repeat
// Continue until all elements are in sorted positions
```

The Heap Sort Process:

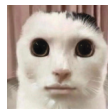
1. **Build heap:** Convert array into max-heap structure
2. **Extract max:** Swap root with last element, reduce heap size
3. **Maintain heap:** Re-heapify after each extraction
4. **Result:** Array becomes sorted in ascending order

We repeatedly extract the maximum element (root of heap) and place it at the end of the array, building the sorted portion from right to left.



67





How Heapify Works:

1. **Find largest** Compare node with its children
2. **Swap if needed** Move larger child up if it's bigger than parent
3. **Recurse** Continue heapifying down the tree
4. **Result** Maintains max-heap property

Heapify ensures that a node and its children satisfy the max-heap property (parent $\geq$ children). If not, it swaps and continues down the tree.

```
void heapify(int[] arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }
    if (largest != i) {
        swap(arr, i, largest);
        heapify(arr, n, largest);
    }
}
```

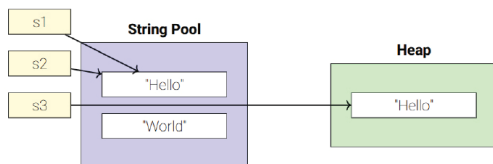
Algorithm	Best	Average	Worst
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Shaker Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Shell Sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

If you ever need to choose (rare!):

- **Selection Sort** Simple, in-place, when memory is limited
- **Bubble Sort** Educational purposes, very small arrays
- **Insertion Sort** Small arrays, nearly sorted data
- **Merge Sort** Stable sorting needed, consistent performance
- **Quick Sort** General purpose, good average performance

The real value: Understanding these algorithms teaches you problem-solving patterns that apply to many other programming challenges!

Strings now!



Key Point String literals share the same object, `new String()` creates separate objects

```
1 String csv = "Alice,25,Engineer";
2 String[] parts = csv.split(",");
3 // Result: ["Alice", "25", "Engineer"]
```

How it works:

- `split()` breaks a string into an array
- Takes a delimiter (what to split on) as parameter
- Returns `String[]` - array of strings
- Original string is unchanged (immutability)

Common use case Processing CSV data, parsing user input

```
= text.replace("World", "Java");
```

```
String formatted = String.format(
    "Name: %s, Age: %d, Score: %.2f",
    name, age, score
);
```

Format Specifiers:

- `%s` - inserts a string
- `%d` - inserts an integer
- `%f` - inserts a floating-point number
- `%.2f` - floating-point with 2 decimal places
- `%n` - platform-independent newline

```
1 StringBuilder sb = new StringBuilder();
2 for (int i = 0; i < 1000; i++) {
3     sb.append(i).append(",");
4 }
5 String result = sb.toString();
```

How it works:

- `StringBuilder` is **mutable** (can be changed)
- `append()` adds to existing buffer (no copying)
- `toString()` creates final string when done
- Only creates **one** string object total

Key Methods:

- `append()` - add to end
- `insert()` - add at specific position
- `delete()` - remove characters
- `reverse()` - reverse the content
- `setCharAt()` - change single character

Operation	String	StringBuilder
Concatenation	Slow (creates new objects)	Fast (modifies buffer)
Memory	High (many temp objects)	Low (single buffer)
Mutability	Immutable	Mutable
Thread Safety	Safe	Not safe
Use Case	Small, fixed strings	Building strings in loops

Pattern	Matches
<code>\d</code>	Any digit (0-9)
<code>\D</code>	Any non-digit
<code>\w</code>	Any word character (a-z, A-Z, 0-9, _)
<code>\W</code>	Any non-word character
<code>\s</code>	Any whitespace (space, tab, newline)
<code>\S</code>	Any non-whitespace
<code>{n}</code>	Exactly n times
<code>{n,m}</code>	Between n and m times

Pattern	Meaning	Example
<code>.</code>	Any single character	<code>a.c</code> matches "abc", "a5c"
<code>*</code>	Zero or more	<code>ab*c</code> matches "ac", "abc", "abbc"
<code>+</code>	One or more	<code>ab+c</code> matches "abc", "abbc"
<code>?</code>	Zero or one	<code>ab?c</code> matches "ac", "abc"
<code>[abc]</code>	Any of a, b, c	<code>[aeiou]</code> matches vowels
<code>[0-9]</code>	Any digit	<code>[0-9]+</code> matches numbers
<code>^</code>	Start of string	<code>^Hello</code> matches start
<code>\$</code>	End of string	<code>end\$</code> matches end



67



wanna call? :3

Using Regex in Java: matches()

```
1 // Simple pattern matching with matches()
2 String text1 = "12345";
3 boolean isDigits = text1.matches("\\d+"); // true
4
5 String text2 = "Hello123";
6 boolean isLetters = text2.matches("[a-zA-Z]+"); // false
7
8 // Phone number pattern (simple)
9 String phone = "555-1234";
10 boolean validPhone = phone.matches("\\d{3}-\\d{4}"); // true
```

Note: In Java strings, backslash must be escaped: \\d

files

```
1 import java.io.File;
2
3 File file = new File("data.txt");
4 boolean exists = file.exists();
```

Key concepts:

- File object represents a file path (not the file itself)
- exists() checks if file actually exists on disk
- File path can be relative or absolute

Important Creating a File object doesn't create the actual file!

```
boolean isFile = file.isFile();
boolean isDir = file.isDirectory();
boolean created = file.createNewFile();
boolean deleted = file.delete();
```

File operations can fail:

- **FileNotFoundException** File doesn't exist
- **IOException** General I/O error (permissions, disk full, etc.)
- **Must handle:** Java requires exception handling for file I/O

Two approaches:

- **try-catch** Handle exceptions in the method
- **throws** Declare that method throws exceptions

Sorted by complexity (simple to advanced):

1. **Files.readAllLines()** Read entire file at once (simplest)
2. **Scanner** Easy to use, good for text files
3. **BufferedReader** More efficient, better for large files

Choose based on

- File size
- Processing needs
- Performance requirements

- **FileWriter** Simple text writing
- **BufferedWriter** Efficient writing with buffering
- **PrintWriter** Convenient formatted output

Write modes

- **Overwrite** Replace existing file content (default)
- **Append** Add to end of existing file

```
try (FileWriter writer = new FileWriter("output.txt")) {
    writer.write("Hello, World!" + System.lineSeparator());
    writer.write("This is line 2" + System.lineSeparator());
    writer.write("And this is line 3" + System.lineSeparator());
    IO.println("File written successfully");
} catch (IOException e) {
    IO.println("Error: " + e.getMessage());
}
```

```
// Replace first match only
String result1 = text.replaceFirst("\\d+", "X");
// Result: "HelloX World456 Java789"
// Replace all matches
String result2 = text.replaceAll("\\d+", "X");

File file = new File("data.txt");

String name = file.getName(); // "data.txt"
String path = file.getAbsolutePath(); // full path
long size = file.length(); // size in bytes

public static Set<String> readCsvFile(String file)
{
    Set<String> words = new HashSet<>();

    try (BufferedReader reader = Files.newBufferedReader(Path.of(file)))
    {
        String line;

        // Skip header
        reader.readLine();

        while ((line = reader.readLine()) != null)
        {
            line = line.trim().toUpperCase();

            if (!line.isEmpty())
            {
                words.add(line);
            }
        }
    } catch (IOException e)
    {
        System.err.println("Error reading file: " + file);
        System.err.println(e.getMessage());
        System.err.println("Make sure the file exists and is accessible. Program will exit.");
        System.exit(1);
    }

    return words;
}
```

```
try {
    List<String> lines = Files.readAllLines(Paths.get("data.txt"));

    for (String line : lines) {
        IO.println(line);
    }
} catch (IOException e) {
    IO.println("Error reading file: " + e.getMessage());
}

try (Scanner scanner = new Scanner(new File("data.txt"))) {
    while (scanner.hasNextLine()) {
        String line = scanner.nextLine();
        IO.println(line);
    }
} catch (FileNotFoundException e) {
    IO.println("File not found: " + e.getMessage());
}
```

- **Windows** CRLF (\r\n)
- **Unix/Linux/Mac** LF (\n)
- **Old Mac** CR (\r)

```
// Append mode (second parameter = true)
try (FileWriter writer = new FileWriter("output.txt", true)) {
    writer.write("This is added to the end" + System.lineSeparator());
} catch (IOException e) {
    IO.println("Error: " + e.getMessage());
}
```

```
try (BufferedWriter writer = new BufferedWriter(
    new FileWriter("output.txt"))) {
    writer.write("Line 1");
    writer.newLine(); // Platform-independent newline
    writer.write("Line 2");
    writer.newLine();
    ...
} catch (IOException e) {
    IO.println("Error: " + e.getMessage());
}
```



67



IO.println("hello wolf");

- **IO.println()**: What we use in this course (simple wrapper)
- **System.out.println()**: Standard Java output to console
- **System.out.print()**: Same but without automatic newline
- **printf() and String.format()**: Covered in Session 8 (Strings)

- **What is Scanner?** Reads formatted input from various sources
- **Why we need it** `System.in` provides raw bytes, Scanner parses them
- **Creating Scanner**: Connect to console input stream (we already did this with files)

```
1 import java.util.Scanner;
2
3 Scanner scanner = new Scanner(System.in);
4
5 // Now we can read user input:
6 String input = scanner.nextLine();
7 IO.println("You entered: " + input);
```

```
boolean hasInt = scanner.hasNextInt();
boolean hasDouble = scanner.hasNextDouble();
boolean hasLine = scanner.hasNextLine();
boolean hasNext = scanner.hasNext();
```

The nextLine() Pitfall

```
1 Scanner scanner = new Scanner(System.in);
2 IO.println("Enter your age:");
3 int age = scanner.nextInt();
4 IO.println("Enter your name:");
5 String name = scanner.nextLine();
6
7 IO.println("Age: " + age); // Works
8 IO.println("Name: " + name); // Empty string!
```

- User types: 25[ENTER] → creates "25\n"
- `nextInt()` reads 25, leaves \n in buffer
- `nextLine()` reads the leftover \n → empty string

```
System.out.println("Normal output"); // Standard output stream
System.err.println("Error message!"); // Error stream (often red)
```

```
// printf() - formatted output (Session 8)
System.out.printf("Player: %s, Score: %d, Accuracy: %.1f%%\n",
    name, score, accuracy * 100);
```

- `next()` - reads until whitespace (one word at a time)
- `nextLine()` - reads until newline (entire line including spaces)
- `nextInt()` - reads and converts to integer
- `nextDouble()` - reads and converts to double
- `nextBoolean()` - reads and converts to boolean
- These methods will crash if input doesn't match expected type!

Two approaches - we'll use both:

Raw Scanner (Standard Java):

```
1 Scanner sc = new Scanner(
    System.in);
2 IO.println("Enter age:");
3 int age = sc.nextInt();
```

IO Helper (Course wrapper):

```
1 IO.println("Enter age:");
2 String input = IO.readLine();
3 int age = Integer.parseInt(
    input);
```

When to use each:

- **IO helpers**: Simpler for basic input, still need to parse manually
- **Raw Scanner**: More control, built-in parsing, real world code

Solution 1: Extra nextLine() to consume newline

Solution 2: Always use nextLine() + parse manually

Solution 3: Use IO helpers (they handle this)

```
1 String input = " abc ";
2
3 // Clean the input before parsing
4 String cleaned = input.trim(); // Remove leading+trailing spaces
5 cleaned = cleaned.toLowerCase(); // Normalize case
6
7 // now parse
8
9 trim() - remove leading/trailing whitespace
10 toLowerCase() - normalize case
11 replace() - remove unwanted characters
```

Best practice: Clean first, then parse safely

- **Definition**: Checking input meets requirements before using it
- **First line of defense**: Catch problems at the boundary
- **Types of validation**
 - **Type validation**: Is it a number? A string? A boolean?
 - **Range validation**: Is $0 \leq \text{age} \leq 120$? Is $1 \leq \text{choice} \leq 5$?
 - **Format validation**: Does email contain @? Is phone 10 digits?
 - **Business rules**: Is username unique? Is password strong enough?

Key principle: Never trust user input!

- **Goal**: Keep asking until valid input received
- **Why do-while?**: Must ask at least once
- **Loop structure**
 1. Ask for input
 2. Validate input
 3. If invalid show error, loop again
 4. If valid break loop, continue program
- **Safety**: Add attempt limit to prevent infinite loops

Real-world use: Menu systems, user registration, game input

Defensive Programming

- **Definition**: Write code expecting the unexpected
- **Goal**: Robust, reliable programs even with bad input or edge cases
- **Connection to input validation**: Validation is defensive programming
- **Key principles**
 - Validate all inputs before processing
 - Handle edge cases explicitly
 - Provide meaningful error messages
 - Fail fast and clearly when something goes wrong
- **Benefits**: Fewer crashes, easier debugging, better user experience
- **Always use**
 - Public methods that others will call
 - Methods that process user input
 - Methods that handle external data (files, network)
 - Critical business logic methods
- **Consider using**
 - Private helper methods in complex classes
 - Methods that perform calculations with many parameters
 - Methods that could fail in unexpected ways
- **Balance**: Don't over-validate simple, internal methods
- **Guard clauses**: Check invalid conditions first and return early
- **Default values**: Provide sensible fallbacks for edge cases
- **Input sanitization**: Clean and normalize data before processing
- **Boundary checking**: Always verify array indices and collection access
- **Resource management**: Ensure files, connections, etc. are properly closed
- **Error logging**: Record problems for debugging without exposing details to users
- **Test invalid inputs**: Negative numbers, null values, out-of-range values
- **Test edge cases**: Zero values, maximum values, boundary conditions
- **Test error messages**: Ensure exceptions have meaningful information
- **Test graceful degradation**: Verify safe default values are returned
- **Test resource cleanup**: Ensure files and connections are properly closed
- **Use assertions**: Verify that validation catches bad inputs



67



recursion
recursion
recursion
recursion
recursion

trust the recursion



Aspect	Recursion	Iteration
Structure	Method calls itself	Loop repeats code
Termination	Base case check	Loop condition
State	Implicit (parameters)	Explicit (variables)
Memory	Call stack grows	Constant stack usage
Readability	Can be elegant	Usually explicit
Debugging	Harder (call stack)	Easier (single frame)

- **What is the stack?** Memory area that stores method call information
 - **Stack frame:** Each method call creates a frame containing:
 - Parameters passed to the method
 - Local variables
 - Return address (where to go when method finishes)
 - **Stack behavior:** Last In, First Out (LIFO)
 - **Recursion uses the stack:** Each recursive call adds a new frame
- Recursion: Benefits

- **Elegant code:** Often more concise and readable
- **Natural fit:** Some problems are inherently recursive
 - Tree traversals
 - Divide-and-conquer algorithms
 - Mathematical definitions
- **Less state management:** Parameters handle state automatically
- **Mathematical clarity:** Mirrors mathematical definitions
- **Divide-and-conquer:** Naturally breaks problems into subproblems

Example: Merge Sort is naturally recursive - divide array, sort halves, merge

Iteration: Benefits

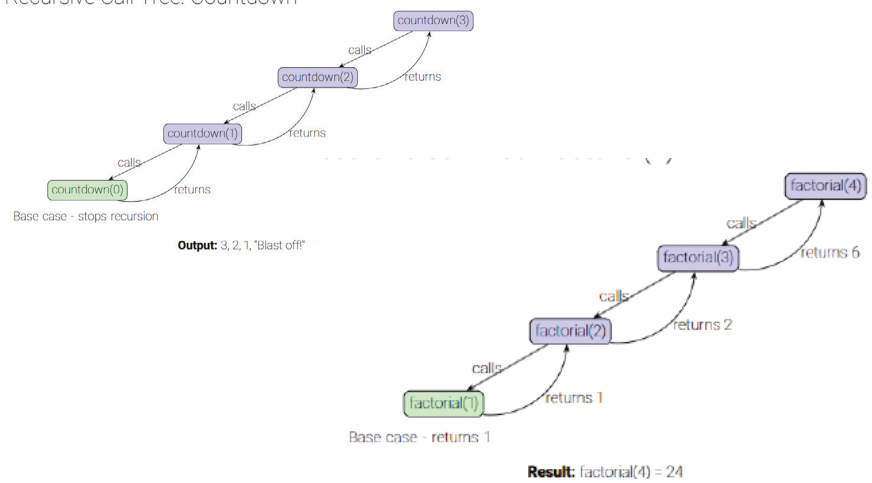
- **Performance:** Usually faster (no method call overhead)
- **Memory efficient:** Constant stack space $O(1)$
- **Easy to debug:** Single stack frame, visible state
- **No stack overflow:** Limited only by available memory
- **Intuitive:** Most developers find loops easier to understand
- **Explicit control:** Clear loop conditions and updates

Example: Iterative factorial is straightforward and efficient

Performance Comparison

Aspect	Recursion	Iteration
Time complexity	Same (usually)	Same (usually)
Space complexity	$O(n)$ stack	$O(1)$ stack
Method call overhead	Yes	No
Memory usage	Higher	Lower
Speed	Slower (overhead)	Faster

Note: For simple problems like factorial, iteration is usually faster. For complex problems like tree traversal, the difference may be negligible.



Both solve the same problem, but with different mental models

Recursive Thinking

- "Solve a smaller problem"
- "Trust the recursion"
- "Combine results"
- Top-down approach

Iterative Thinking

- "Repeat this action"
- "Update state each time"
- "Accumulate results"
- Bottom-up approach

Recursive Factorial(n)

- **Stack frames:** n frames
- **Memory per frame:** ~ 24 bytes (parameters, return address)
- **Total stack memory:** $O(n)$
- **Example:** factorial(1000) needs ~ 24 KB stack

Key insight: Recursion uses more memory because it stores state for each recursive call

Iterative Factorial(n)

- **Stack frames:** 1 frame
- **Memory per frame:** ~ 16 bytes (variables)
- **Total stack memory:** $O(1)$
- **Example:** factorial(1000) needs ~ 16 bytes

Recursion: Drawbacks

- **Stack overhead:** Each call uses stack memory
- **Performance cost:** Method call overhead
- **Stack overflow risk:** Deep recursion can crash
- **Harder to debug:** Multiple stack frames to trace
- **Less intuitive:** Some developers find it harder to understand
- **Memory usage:** $O(n)$ stack space for depth n

Example: Recursive Fibonacci is very inefficient due to repeated calculations

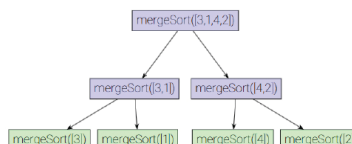
Iteration: Drawbacks

- **More verbose:** Sometimes requires more code
- **State management:** Need to manually track variables
- **Less natural:** Some problems don't fit iterative patterns well
- **Complex logic:** Can become hard to read with nested loops
- **Error-prone:** Off-by-one errors, incorrect loop conditions

Example: Iterative tree traversal requires explicit stack management

Base cases:
Single elements
(already sorted)

Recursive:
Divide and
conquer



67



Why Recursion Fits Divide-and-Conquer

- **Natural decomposition:** Problems break into identical subproblems
- **Base case clarity:** Smallest problem is easy to solve
- **Combination step:** Results from subproblems combine naturally
- **Code elegance:** Recursive code mirrors the problem structure

Examples of divide-and-conquer:

- Merge Sort: Sort halves, then merge
- Quick Sort: Partition, then sort partitions
- Binary Search: Search half, then search that half

Decision Framework: When to Use Recursion?

Use recursion when:

- **Problem is naturally recursive:** Tree/graph structures, divide-and-conquer
- **Mathematical definition is recursive:** Factorial, Fibonacci, mathematical sequences
- **Code clarity matters more than performance:** Elegant solution is worth small overhead
- **Problem depth is limited:** Won't cause stack overflow
- **State management is complex:** Recursion handles it automatically

Examples Tree traversal, Merge Sort, Quick Sort, parsing nested structures

Decision Framework: When to Use Iteration?

Use iteration when:

- **Performance is critical:** Need maximum speed
- **Memory is limited:** Can't afford stack overhead
- **Problem depth is very large:** Risk of stack overflow
- **Simple repetitive task:** No need for complex decomposition
- **Debugging ease matters:** Need to trace execution easily

Examples: Summing arrays, factorial, Fibonacci, simple loops

Hybrid Approaches

- **Recursive with iterative base:** Use recursion for structure, iteration for base cases
- **Iterative with recursive thinking:** Use iterative implementation of recursive algorithm
- **Memoization:** Cache recursive results to avoid repeated calculations
- **Tail recursion:** Optimize recursive calls that don't need stack frames

Example: Iterative Merge Sort (possible but more complex) or Memoized Fibonacci (recursive but efficient)

Recursion in Practice

- File system traversal
- XML/JSON parsing
- Compiler parsing
- Tree algorithms
- Graph algorithms

Iteration in Practice

- Array processing
- Database queries
- Game loops
- Data processing pipelines
- Simple algorithms

Recursion-Friendly Problems

- **Tree structures:** Each node has similar structure
- **Divide-and-conquer:** Problem splits into similar subproblems
- **Backtracking:** Need to explore multiple paths
- **Nested structures:** Data with recursive definition
- **Mathematical recursion:** Defined recursively in math

Key pattern: Problem structure mirrors recursive structure
Problem Characteristics: Iteration-Friendly

Iteration-Friendly Problems

- **Linear processing:** Process items in sequence
- **Accumulation:** Build result step by step
- **Simple repetition:** Same action multiple times
- **State machines:** Clear state transitions
- **Performance-critical:** Need maximum efficiency

Key pattern: Problem is sequential, not hierarchical



67



"i don't debug, i don't analyze code, I press compile and if my code doesn't work I KILL MYSELF"



- **Syntax errors:** Code violates language rules → won't compile
- **Runtime errors:** Code compiles but crashes during execution
- **Logic errors:** Code runs but produces incorrect results

Three Levels of Testing: Overview

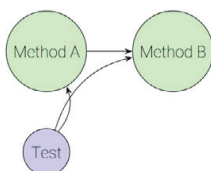
Unit Tests

- Test one method
- Fast execution
- Many tests
- Isolated



Integration Tests

- Test components together
- Medium speed
- Some tests
- Test interactions



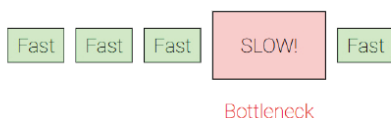
System Tests

- Test entire app
- Slow execution
- Few tests
- End-to-end

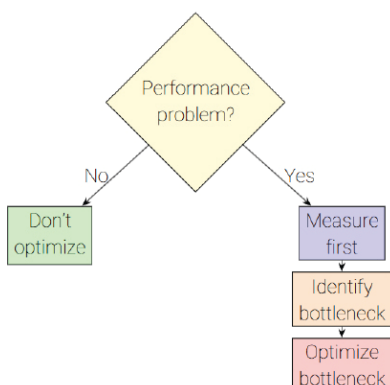


(Overview next semester)

- **What is a bottleneck?** Part of code that slows down entire program
- **How to find:** Profiler shows which methods take longest
- **Hot spots:** Methods that consume most execution time
- **80/20 rule:** Often 20% of code takes 80% of time



Key insight Optimize the bottleneck, not the fast parts

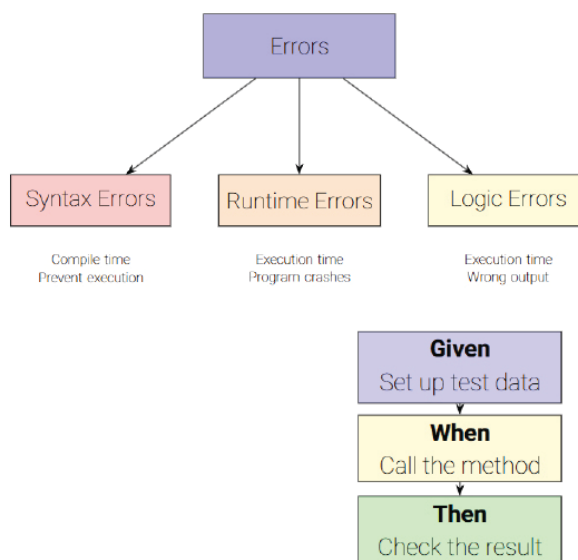


Key principles:

- Don't optimize prematurely
- Measure first, then optimize
- Optimize bottlenecks, not everything

Remember

Profiling tells you *where* to optimize, not *when* to optimize



The Pattern:

1. **Given:** Prepare inputs, set up test data (preconditions)
2. **When:** Call the method being tested (the action)
3. **Then:** Verify the result is correct (the assertion)

Common Performance Issues

Inefficient Algorithms

- Recursion when iteration is possible (most times)
- Nested loops when single loop works
- Example: Bubble sort vs merge sort

Unnecessary Computations

- Recalculating same value multiple times
- Not caching results
- Example: Computing factorial(5) multiple times

Key insight Profiling reveals these issues so you can fix them



67



Memory Model: 2D Arrays on Stack and Heap

Stack Variables

Variable	Value
matrix	0x1000

```
1 static void example() {
2     int[] [] matrix = new int
3     [3] [4];
4     matrix[1][2] = 42;
5 }
6 }
```

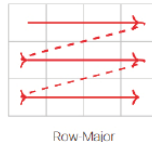
Heap Objects

Address	Object
0x1000	Main Array: [ref0, ref1, ref2]
0x2000	Row 0: [0, 0, 0, 0]
0x3000	Row 1: [0, 0, 42, 0]
0x4000	Row 2: [0, 0, 0, 0]

Concept Row by Row.

- Outer Loop: **row**
- Inner Loop: **col**
- Cache Friendly** (Contiguous memory)

```
1 for (int r = 0; r < rows; r++) {
2     for (int c = 0; c < cols; c++) {
3         print(matrix[r][c]);
4     }
5     println();
6 }
```



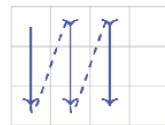
Row-Major

Traversing: Column-Major

Concept Column by Column.

- Outer Loop: **col**
- Inner Loop: **row**
- Use Case**: Vertical checks (Connect 4)

```
1 for (int c = 0; c < cols; c++) {
2     for (int r = 0; r < rows; r++) {
3         print(matrix[r][c]);
4     }
5 }
```

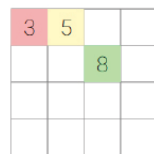


Column-Major

Three Types of Cells:

- Corner**: 3 neighbors (e.g., top-left)
- Edge**: 5 neighbors (e.g., top row, not corner)
- Interior**: 8 neighbors (middle of grid)

Solution Always validate before accessing!

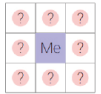


Neighbor counts

Pattern: Neighbor Checking (The Minesweeper Problem)

Problem: For a cell at $[r][c]$, check its 8 neighbors.

- Top-Left: $[r-1][c-1]$
- Top: $[r-1][c]$
- ...and so on.



Challenge: Boundary Checks!

- If $r=0$, we can't check $r-1$ (IndexOutOfBounds)
- Must check `isValid(r, c)` before accessing.

Three Types of Cells:

- Corner**: 3 neighbors (e.g., top-left)
- Edge**: 5 neighbors (e.g., top row, not corner)
- Interior**: 8 neighbors (middle of grid)

Solution: Always validate before accessing!

The Standard Pattern: Create a helper method to check boundaries.

```
1 // Check if (r, c) is a valid position in the grid
2 static boolean isValid(int[] [] grid, int r, int c) {
3     // Check row bounds
4     if (r < 0 || r >= grid.length) return false;
5     // Check column bounds (use first row's length)
6     if (c < 0 || c >= grid[0].length) return false;
7     return true;
8 }
```

Concept Filling a region (like Paint Bucket tool).

- Start at a pixel.
- Spread to neighbors if they are the same color.
- Preview**: This uses **Recursion**!



Transpose: Swap Rows and Columns.

Count neighbors in a Minesweeper-style grid:

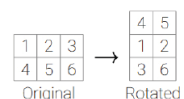
```
1 int countNeighbors(int[] [] grid, int r, int c) {
2     int count = 0;
3     // Check all 8 directions
4     int[] dr = {-1, -1, -1, 0, 0, 1, 1, 1}; // row deltas
5     int[] dc = {-1, 0, 1, -1, 1, -1, 0, 1}; // col deltas
6
7     for (int i = 0; i < 8; i++) {
8         int newR = r + dr[i];
9         int newC = c + dc[i];
10        // Only check if valid position
11        if (isValid(grid, newR, newC) && grid[newR][newC] == 1) {
12            count++;
13        }
14    }
15    return count;
16 }
```

Key The `isValid()` check prevents `ArrayIndexOutOfBoundsException`!

```
1 int[] [] transpose(int[] [] matrix) {
2     int h = matrix.length; int w = matrix[0].length;
3     int[] [] result = new int[w][h]; // Dimensions swapped!
4
5     for (int r = 0; r < h; r++) {
6         for (int c = 0; c < w; c++) {
7             result[c][r] = matrix[r][c]; // Swap indices
8         }
9     }
10    return result;
11 }
```

Concept Rotate a grid clockwise by 90 degrees.

```
int[] [] rotate90(int[] [] grid) {
    int rows = grid.length;
    int cols = grid[0].length;
    int[] [] rotated = new int[cols][rows];
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            // Rotate: (r,c) -> (c, rows-1-r)
            rotated[c][rows-1-r] = grid[r][c];
        }
    }
    return rotated;
}
```



Key Insight Dimensions swap! A 3x2 grid becomes 2x3 after rotation.



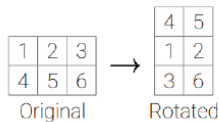
67



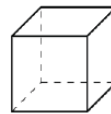
Concept Rotate a grid clockwise by 90 degrees.

```
int[][] rotate90(int[][] grid) {
    int rows = grid.length;
    int cols = grid[0].length;
    int[][] rotated = new int[cols][rows];

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            // Rotate: (r,c) -> (c, rows-1-r)
            rotated[c][rows - 1 - r] = grid[r][c];
        }
    }
    return rotated;
}
```



- **Concept** An array of 2D arrays.
- **Visual**: A Cube, or a Stack of Pages (Book).
- **Syntax** `int[][][] cube = new int[depth][rows][cols];`
- **Access** `cube[z][y][x]` (or `[layer][row][col]`)

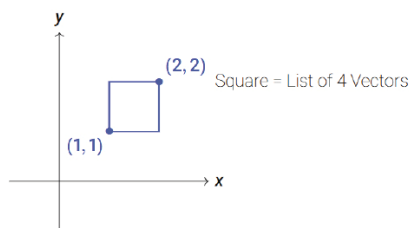


Key Insight: Dimensions swap! A 3x2 grid becomes 2x3 after rotation.

- **3D Space**: Voxel engines (like Minecraft chunks)
- **Time**: A sequence of 2D board states (History of a chess game)
- **Color Images**:
 - 2D Grid of Pixels
 - Each Pixel has 3 values: Red, Green, Blue
 - Structure: `[height][width][3]`

Matrix Multiplication (Conceptual): Row of A · Column of B (Dot Product). Requires **Triple Nested Loop**

```
1 // Matrix Multiplication: C = A * B
2 for (int i = 0; i < rows; i++) {
3     for (int j = 0; j < cols; j++) {
4         for (int k = 0; k < common; k++) {
5             result[i][j] += A[i][k] * B[k][j];
6         }
7     }
8 }
```



- **Goal**: Move the object by (d_x, d_y) .
- **Math**: Vector Addition.
- **Formula** $v_{new} = v_{old} + \text{translation}$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

Move Right by 2

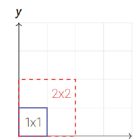


Transformation 2: Scaling (Resizing)

- **Goal**: Make the object bigger or smaller.
- **Math**: Matrix Multiplication.
- **Matrix** $\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

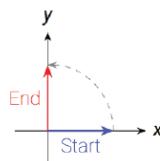
Double the size (2x, 2y)



- **Goal**: Rotate around the origin (0,0)
- **Math**: Matrix Multiplication.
- **Matrix** $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$

$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Rotate 90° Counter-Clockwise



Concept An image is just a grid of pixels.

- Grayscale: 0 (Black) to 255 (White)
- **Task**: Invert the image (Negative)

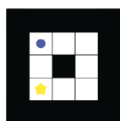
```
1 // Invert Image
2 for (int r = 0; r < height; r++) {
3     for (int c = 0; c < width; c++) {
4         int original = image[r][c];
5         image[r][c] = 255 - original;
6     }
7 }
```

Deep Dive: Mazes & Game Logic

Grid Representation

- 0 = Path, 1 = Wall, 2 = Player, 9 = Goal

```
1 int[][] maze = {
2     {1, 1, 1, 1, 1},
3     {1, 2, 0, 0, 1},
4     {1, 1, 1, 0, 1},
5     {1, 9, 0, 0, 1},
6     {1, 1, 1, 1, 1}
7 };
8
9 boolean isValidMove(int r, int c) {
10    // Check boundaries
11    if (r < 0 || r >= rows || c < 0 || c >= cols)
12        return false;
13    // Check wall
14    if (maze[r][c] == 1) return false;
15    return true;
16 }
```



Concept Recursion is great for exploring grids (Pathfinding).

```
1 boolean solveMaze(int r, int c) {
2     // 1. Base Case: Out of bounds or Wall
3     if (!isValid(r, c) || isWall(r, c)) return false;
4     // 2. Base Case: Reached Goal!
5     if (isGoal(r, c)) return true;
6
7     // 3. Recursive Step: Try all 4 directions
8     markVisited(r, c);
9     if (solveMaze(r+1, c)) return true; // Down
10    if (solveMaze(r, c+1)) return true; // Right
11    // ... Up and Left
12    return false; // Backtrack
13 }
14 }
```

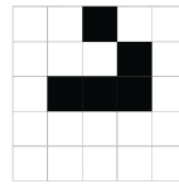


67



The Rules of Life:

- Grid of cells: Alive (1) or Dead (0).
- Underpopulation:** < 2 neighbors $\rightarrow$ Dies.
- Overpopulation:** > 3 neighbors $\rightarrow$ Dies.
- Survival:** 2 or 3 neighbors $\rightarrow$ Lives.
- Reproduction:** Exactly 3 neighbors $\rightarrow$ Becomes Alive.

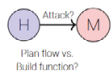


The "Glider"

Why?: It's Turing Complete! You can build a computer inside it.

Before We Begin: Consider These Scenarios

1. Building a Game

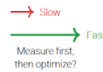


- Game:** Top-down (Plan) vs Bottom-up (Function)
- Data:** Brute force search vs Sort-then-Search
- Performance:** Premature optimization vs Profiling

2. Organizing Data



3. Performance



Consider:

- Speed?
- Memory usage?
- Code clarity?
- Maintainability?
- Ease of debugging?

The answer:

- It depends on context!
- Different problems need different solutions
- Understanding trade-offs is key

Today we'll learn frameworks for making these decisions

Review: Design Approaches We've Learned

- Top-Down Design:** Start with the big picture, break it down
- Bottom-Up Design:** Start with small pieces, build up
- Both approaches:** Used together in real-world development
- Connection:** These are problem-solving paradigms—systematic ways to think about problems



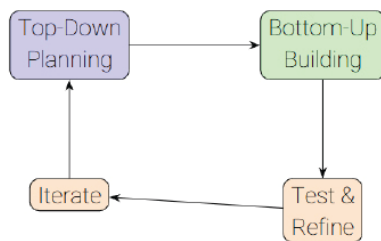
67



Aspect	Top-Down	Bottom-Up
When to use	Starting new projects, understanding requirements	Debugging, building reusable components
Best for	Planning, communication, breaking down problems	Testing, learning building blocks, fixing bugs
Thinking style	"What should this do?"	"What can I build with this?"
Risk	Over-planning, missing details	Building wrong pieces, unclear direction

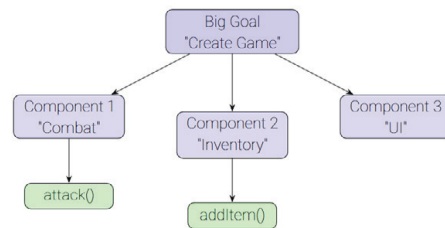
Real-world development:

- **Top-down for planning:** Understand the big picture first
- **Bottom-up for implementation:** Build and test small pieces
- **Iterative refinement:** Go back and forth between approaches



Top-Down Design: Recap

1. Start with the **big goal** (very abstract)
2. Break it into **major components** (less abstract)
3. Break each component into **smaller steps** (more specific)
4. Keep breaking until you reach **implementable code**

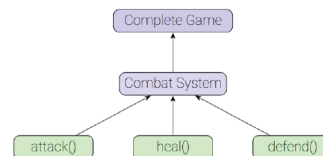


Page: 7 / 13

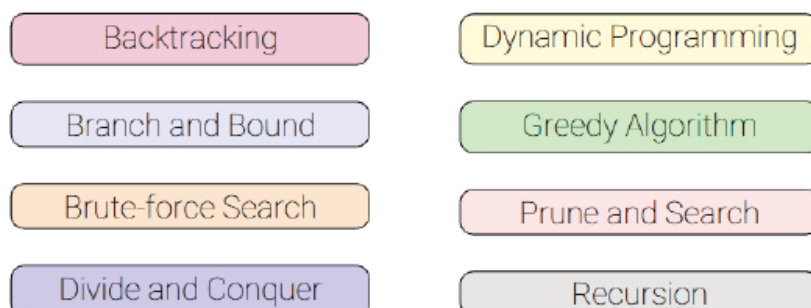
Bottom-Up Design: Recap

How it works:

1. Start with **small, working pieces** (very specific)
2. Combine pieces into **larger components** (less specific)
3. Combine components into **complete systems** (more abstract)
4. Test each level as you build up



- **Definition:** Reusable problem-solving templates or strategies
- **Like toolboxes:** Each paradigm is a set of techniques for specific problem types
- **Why learn them:**
 - Recognize patterns in problems
 - Choose the right approach quickly
 - Understand existing code better
 - Communicate solutions clearly



age 11 | 1



67



Divide-and-Conquer Strategy

Core idea: Break problem into smaller subproblems, solve recursively, combine results

Three steps:

1. **Divide:** Split problem into smaller instances
2. **Conquer:** Solve subproblems recursively (or directly if small enough)
3. **Combine:** Merge solutions into final answer

You've seen this before:

- **Merge Sort** (Session 07): Divide array, sort halves, merge
- **Binary Search** (Session 07): Divide search space in half
- **Recursion** (Session 10): Natural fit for divide-and-conquer

Prune and Search Strategy

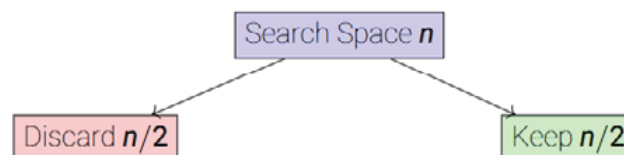
Core idea: Like Divide and Conquer, but we throw away parts of the problem that we know don't contain the solution.

Key difference from Divide and Conquer:

- **Divide and Conquer:** Solves *all* subproblems (e.g., Merge Sort)
- **Prune and Search:** Solves *only one* subproblem and discards the rest (e.g., Binary Search)

Example: Binary Search

- Is target > mid? Yes → Prune left half, search right.
- Search space shrinks: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$



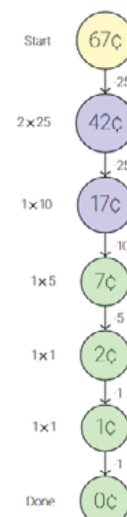
Greedy Strategy

Core idea: Make the locally optimal choice at each step, hoping it leads to global optimum

- **No backtracking:** Once a choice is made, it's final
- **Fast:** Usually simple and efficient
- **Not always optimal:** May miss better solutions

Simple example: Coin Change (Greedy)

- Goal: Make change with fewest coins
- Strategy: Always use largest coin possible
- Example: 67 cents = 2 quarters + 1 dime + 1 nickel + 2 pennies



67



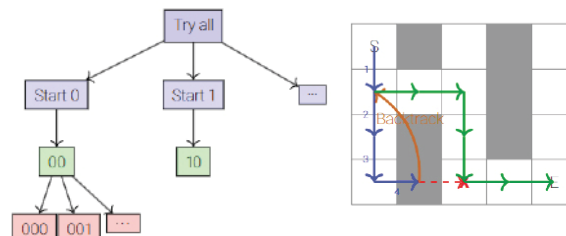
Brute Force Strategy

Core idea: Try all possible solutions, pick the best one

- **Guaranteed to find solution:** If one exists, brute force will find it
- **Simple to implement:** Just try everything
- **Very slow:** Exponential time complexity for many problems

Example: Finding Password

- For 4-digit PIN: Try 0000, 0001, 0002, ..., 9999
- Guaranteed to find it, but may take up to 10,000 tries



Steps:

1. Go South (path blocked later)
2. Hit Wall/Dead End
3. **Backtrack** to previous intersection
4. Try East (Success!)

Backtracking Strategy

Core idea: "Smart Brute Force" - build a solution step by step, but abandon ("backtrack") as soon as we know a path won't work.

The Algorithm:

1. **Choose:** Make a choice (e.g., go North)
2. **Explore:** Recursively try to solve the problem with this choice
3. **Un-choose:** If it didn't work, undo the choice (backtrack) and try the next option

Perfect for:

- Mazes and Pathfinding
- Puzzles (Sudoku, Crosswords)
- Combinatorial problems (N-Queens)

Backtracking: Java Example (Maze)

```
1 boolean solveMaze(int r, int c) {
2     // 1. Base Cases
3     if (!isValid(r, c) || isWall(r, c) || visited[r][c]) return false;
4     if (isExit(r, c)) return true; // Found it!
5     // 2. Choose
6     visited[r][c] = true;
7     // 3. Explore (Recursion)
8     if (solveMaze(r + 1, c)) return true; // South
9     if (solveMaze(r, c + 1)) return true; // East
10    if (solveMaze(r - 1, c)) return true; // North
11    if (solveMaze(r, c - 1)) return true; // West
12    // 4. Un-choose (Backtrack)
13    return false;
14 }
```

Page 23 | }

Branch and Bound Strategy

Core idea: Optimizing Backtracking. Before exploring a path (branch), calculate if it *could* possibly lead to a better solution than what we already have. If not, cut it off (bound).

Analogy: Finding the shortest path.

- You found a path of length 10km.
- You start exploring a new path and traveled 11km already.
- **Bound:** Stop! Even if the rest is 0km, 11 > 10.
- **Branch:** Don't explore this path further.

Result: Much faster than standard Backtracking because it prunes large sections of the search tree.



67



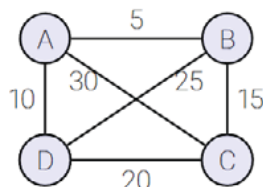
The Traveling Salesperson Problem (TSP)

The Problem:

- Given a list of cities and the distances between each pair of cities.
- **Goal:** Find the shortest possible route that visits each city exactly once and returns to the origin city.

Why it's interesting:

- Looks simple, but is extremely hard to solve optimally for many cities.
- **Complexity:** $O(n!)$ (Factorial) using Brute Force.
- ideal candidate for improvements like **Branch and Bound**.

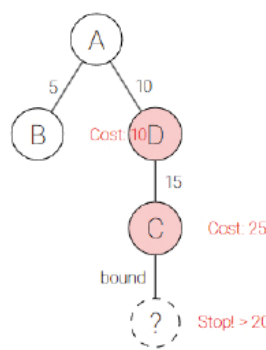


Branch and Bound: Example (TSP)

Goal: Visit 4 cities (A, B, C, D) with minimum total distance.

Status:

- **Current Best Solution:** 20km (A → B → C → D → A)
- **Currently Exploring:** Path A → D → C...
- **Current Cost:** 25km



Dynamic Programming: Fibonacci

```

1 long[] memo = new long[100]; // Cache
2
3 long fib(int n) {
4     if (n <= 1) return n;
5
6     // Check cache first!
7     if (memo[n] != 0) return memo[n];
8
9     // Calculate and save
10    memo[n] = fib(n-1) + fib(n-2);
11    return memo[n];
12 }
    
```

Decision:

- Cost (25) > Best (20)
- **PRUNE!** Stop exploring this branch.
- We don't care how far it is back to A.

Dynamic Programming (DP)

Core idea: Optimization of plain recursion. If a problem has **Overlapping Subproblems**, solve each subproblem once and save the result.

Techniques:

- **Memoization** (Top-Down): Recursive + Caching. "Have I solved this before?"
- **Tabulation** (Bottom-Up): Iterative table filling. "Build up from 0."

Without DP ($O(2^n)$):

- fib(5) calls fib(4), fib(3)
- fib(4) calls fib(3), fib(2)
- fib(3) calculated twice!

With DP ($O(n)$):

- Calculate fib(3) once
- Save result in array/map
- Next time, just look it up

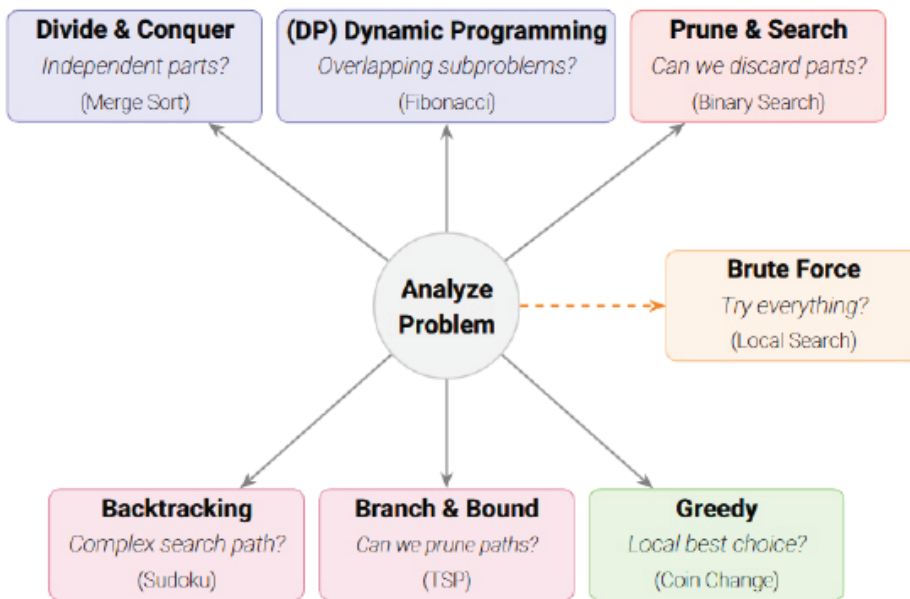
Strategy Comparison

Strategy	When to Use	Pros	Cons
Divide and Conquer	Problems that break into subproblems	Efficient, elegant, natural recursion	Requires combining step
Prune and Search	Search spaces where parts can be discarded	Extremely fast ($O(\log n)$)	Requires sorted/structured data
Greedy	Optimization problems with local choices	Fast, simple	May not find optimal solution
Brute Force	Small problem spaces	Guaranteed correct	Very slow ($O(2^n)$, $O(n!)$)
Backtracking	Complex search spaces (mazes)	Finds solution, better than Brute Force	Can still be slow
Branch and Bound	Optimization problems	Prunes search tree, faster than Backtracking	Worst case still exponential
Dynamic Programming	Overlapping subproblems	Optimal, fast ($O(n)$)	Uses extra space ($O(n)$)



67





What happens when your program is too slow?

Real-world impact:

- **Search engines:** Must return results in milliseconds
- **Games:** Need (at least) 60 frames per second (16ms per frame)
- **Mobile apps:** Slow apps get deleted
- **User experience:** Waiting feels like forever

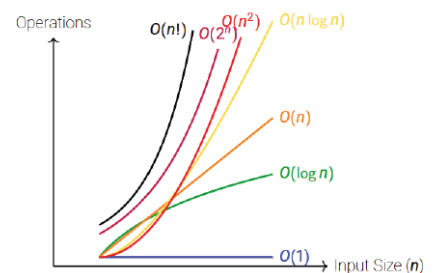
But also:

- Not everything needs to be optimized
- Premature optimization is the root of all evil. (Donald Knuth)
- Understanding trade-offs is more important than perfect performance

- **$O(1)$ Constant:** Array access. Instant.
- **$O(\log n)$ Logarithmic:** Binary Search. Very fast, even for huge data.
- **$O(n)$ Linear:** Simple loop. Grows evenly with data.
- **$O(n \log n)$ Log-Linear:** Good sorting (Merge Sort). Standard for sorting.
- **$O(n^2)$ Quadratic:** Nested loops. Slow for large data.
- **$O(2^n)$ Exponential:** Recursion without pruning. Unusable for large n .

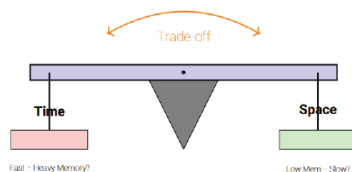
What is Big O? It describes the **Upper Bound** of an algorithm's growth.

- Usually represents the **Worst Case** scenario.
- "My code will never be slower than this."
- (Average case is often similar, but Big O guarantees the limit).



Complexity	Meaning	Example
$O(1)$	Constant time - same time regardless of input	Array access
$O(\log n)$	Logarithmic - doubles input, adds one step	Binary search
$O(n)$	Linear - time grows with input size	Linear search
$O(n \log n)$	Linearithmic - efficient sorting	Merge Sort
$O(n^2)$	Quadratic - nested loops	Bubble Sort

Core concept Often you can use more memory to save time, or more time to save memory



Examples:

- **Merge Sort:** Uses $O(n)$ extra space for $O(n \log n)$ time
- **Recursive vs Iterative:** Recursion uses stack space, iteration uses constant space
- **Caching:** Store results to avoid recomputation (more memory, less time)

Key insight As n grows, the difference becomes huge!

- For $n=1,000,000$: $O(n) = 1\text{M}$ operations, $O(n^2) = 1$ trillion operations
- For $n=1,000,000$: $O(\log n) = 20$ operations, $O(n) = 1\text{M}$ operations

Recall from Session 10: Fibonacci example

Recursive Fibonacci

- **Time:** $O(2^n)$ - exponential!
- **Space:** $O(n)$ - call stack
- **Problem:** Recalculates same values

Iterative Fibonacci

- **Time:** $O(n)$ - linear!
- **Space:** $O(1)$ - constant
- **Solution:** Stores previous values



Key insight Iterative version trades a bit of code complexity for huge performance gain



67



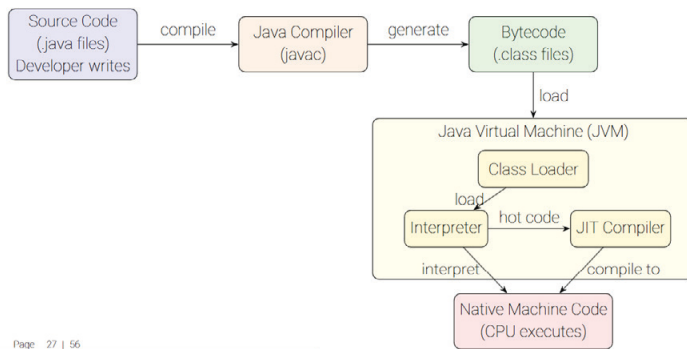
Common Pitfall: Choosing Wrong Paradigm

Examples of mismatched strategies:

- **Using brute force** for large search spaces
 - Problem: Trying all combinations for 20 items
 - Result: Millions of operations, too slow
 - Better: Use divide-and-conquer or greedy
- **Using divide-and-conquer** for simple linear problems
 - Problem: Summing an array
 - Result: Unnecessary complexity
 - Better: Simple loop ($O(n)$)
- **Using greedy** when optimal solution needed
 - Problem: Need guaranteed best solution
 - Result: May find suboptimal answer
 - Better: Use exhaustive search or dynamic programming

Solution: Understand problem characteristics, then choose matching strategy

From Source to Running Program



Page: 27 | 56

Syntax = Grammar rules (how to write it correctly)

Semantics = Meaning (what it does when it runs)

Java: Strengths and Weaknesses

Strengths:

- **Platform Independence:** "Write once, run anywhere"
- **Strong Typing:** Catches errors at compile time
- **Memory Management:** Automatic garbage collection
- **Rich Ecosystem:** Extensive libraries and frameworks
- **Object-Oriented:** we continue using it next semester

Weaknesses:

- **Verbose:** More code required for simple tasks
- **Performance Overhead:** JVM adds some overhead
- **Memory Usage:** Higher memory consumption than C/C++
- **Startup Time:** Slower initial execution

Compiler (javac) Translates source code (**.java**) to bytecode (**.class**)

Bytecode Platform-neutral instructions stored in **.class** files

JVM Java Virtual Machine that loads and executes bytecode

Class Loader JVM component that loads classes into memory

Interpreter JVM component that executes bytecode step-by-step

JIT Compiler Just-In-Time compiler that converts hot code to native machine code

Native Code Machine-specific instructions that the CPU executes directly

```
public static long factorial(long n)
{
    if (n <= 1)
    {
        return 1;
    }
    return n * factorial(n - 1);
}
```

factorial(4)

```
-> 4 * factorial(3) | n = 4
-> 3 * factorial(2) | n = 3
-> 2 * factorial(1) | n = 2
-> 1 | n = 1
```

```
-> return 1
-> return 2 * 1
-> return 3 * 2
-> return 4 * 6
```

-> return 24

Bitwise Operators

- **&** (AND): bit-by-bit AND operation
- **|** (OR): bit-by-bit OR operation
- **^** (XOR): bit-by-bit exclusive OR
- **~** (NOT): inverts all bits
- **<<** **>>** (shift operators)

```
1    int flags = 0b1010;        // binary: 1010 (decimal 10)
2    int mask = 0b1100;         // binary: 1100 (decimal 12)
3    int result1 = flags & mask; // 0b1000 (decimal 8)
4    int result2 = flags | mask; // 0b1110 (decimal 14)
5    int result3 = flags ^ mask; // 0b0110 (decimal 6)
6    int result4 = ~flags;       // 0b0101 (decimal -11)
7    int shifted = flags << 1;   // 0b10100 (decimal 20)
```



67



Page Overview

Page 1 - Java basics overview

Primitive types, ranges, defaults, operators, short-circuiting, autoboxing, loop control (break / continue), JVM memory areas

Page 2 - Stack vs Heap & method design

Stack frames, heap objects, memory management, method signatures, testing strategies, assertions, arrays (.length)

Page 3 - Searching & Sorting fundamentals

Binary vs linear search, complexity intuition, stability, in-place vs out-of-place sorts, Bubble / Selection / Insertion

Page 4 - Sorting algorithms (advanced)

Shaker Sort, Merge Sort, Quick Sort, Shell Sort, Heap Sort with core concepts and code

Page 5 - Strings in Java

String pool, immutability, StringBuilder, common methods, formatting, regex basics

Page 6 - Files & Regular Expressions

File I/O, BufferedReader, CSV reading, error handling, regex usage, file writing

Page 7 - Input/Output & Defensive Programming

Scanner usage, IO helpers, input pitfalls, guard clauses, validation, error handling

Page 8 - Recursion vs Iteration

Recursive thinking, call stack, benefits & drawbacks, factorial examples, performance comparison

Page 9 - When to use recursion or iteration

Decision frameworks, hybrid approaches, real-world use cases

Page 10 - Testing, Debugging & Performance

Error types, testing levels, profiling, bottlenecks, optimization principles

Page 11 - 2D Arrays & Grid Algorithms

Row/column traversal, neighbor checking, Minesweeper logic, transpose

Page 12 - Matrices, Geometry & Game Logic

Rotation, vectors, transformations, matrices, maze/pathfinding concepts

Page 13 - Cellular Automata & Design Review

Game of Life, performance vs clarity trade-offs, design approaches recap

Page 14 - Problem-Solving Paradigms

Top-down vs bottom-up, reusable strategies, algorithmic toolboxes

Page 15 - Divide & Conquer, Prune & Search, Greedy

Binary search, merge sort, coin change example

Page 16 - Brute Force, Backtracking & Branch and Bound

Maze solving, optimization through pruning

Page 17 - Traveling Salesperson Problem & Dynamic Programming

Branch & bound example, memoization, DP vs recursion

Page 18 - Algorithm Analysis & Big-O

Complexity classes, trade-offs, recursion vs iteration performance

Page 19 - Common Pitfalls & Java Internals

Choosing the right paradigm, compilation pipeline, JVM, bitwise operators



67

